

## 第二章 词法分析

# 本 课 目 标

---



理解分词的基本概念和相关方法



理解关键词提取的基本概念和相关方法



理解词性标注的基本概念和相关方法



理解命名实体识别的基本概念和相关方法

# 课程目录

Course catalogue

1/ 什么是词法分析

2/ 分词

3/ 关键词提取

4/ 词性标注

5/ 命名实体识别

## 什么是词法分析？

**词法分析**，是利用计算机对自然语言的形态 (morphology) 进行分析，判断词的结构和类别等，是自然语言处理中的关键组成部分。针对某种语言的词法分析与语言本身的特性相关，不能一概而论。对于中文来说，词法分析的主要任务首先是能正确地把一串连续的字符切分成一个一个的词 (Token)，然后能够正确地判断每个词的词性，以便于后续的句法分析的实现。中文分词和词性标注等词法分析任务一般被统称为**中文词法分析**。

# 课程目录

Course catalogue

1/ 什么是词法分析

2/ 分词

3/ 关键词提取

4/ 词性标注

5/ 命名实体识别

# 中文分词技术分类

## 中文分词技术分类

分词算法可分为三大类：

- ① 基于字典、词库匹配的分词方法；（字符串匹配、机械分词法）
- ② 基于词频度统计的分词方法；
- ③ 基于知识理解的分词方法。

第一类方法应用词典匹配、汉语词法或其它汉语语言知识进行分词，如：最大匹配法、最小分词方法等。这类方法简单、分词效率较高，但汉语语言现象复杂丰富，词典的完备性、规则的一致性等问题使其难以适应开放的大规模文本的分词处理。

第二类基于统计的分词方法则基于字和词的统计信息，如把相邻字间的信息、词频及相应的共现信息等应用于分词，由于这些信息是通过调查真实语料而取得的，因而基于统计的分词方法具有较好的实用性。

# 基于词典的分词

## 正向最大匹配分词

(Forward Maximum Matching, FMM)

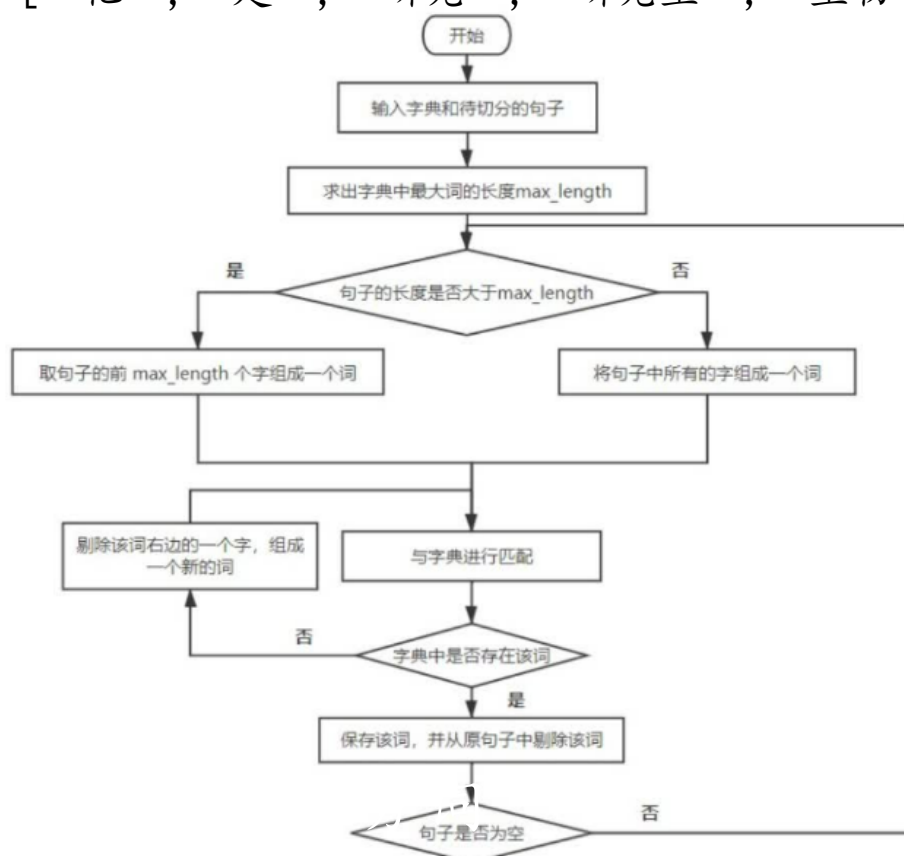
是一种基于词典的分词方法，其核心思想是从左到右尽可能匹配最长的词。

给定一个句子：

“他是研究生物化学的”。

给定一个词典：

[“他”，“是”，“研究”，“研究生”，“生物”，“物化”，“化学”，“学”，“的”]。



先获得词典中词的最大长度 $m$ ，这个例子中 $m$ 为3；

给字符串初始位置一个指针 $pi$ ，即在“他”的位置；

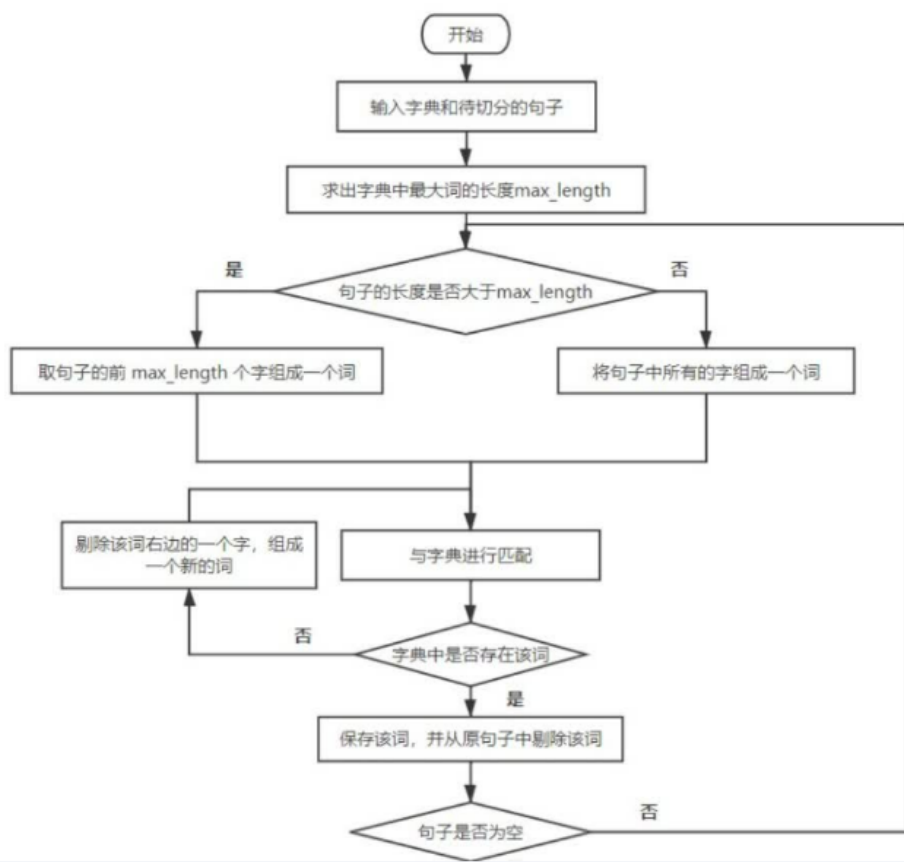
从当前指针起取 $m$ 个字作为词，即“他是研”；

选出的词如果在词典中，就在词的后面进行划分，然后指针移动到这个词后面的一个字，

如果选出的词不在词典中，就把选出的词长度减一（即 $m-1$ ），“他是研”就变成了“他是”，然后再进行此步骤的操作。

# 基于词典的分词

## 正向最大匹配分词



算法流程:

输入: 字符串  $s$

过程:

1. 令指针  $pi$  指向  $s$  的初始位置
  2. repeat
  3. 计算当前指针  $pi$  到字符串末端的字数 (即未被切分字符串的长度)  $n$
  4. 令  $m$  = 词典中最长单词的字数, 如果  $n < m$
  5. 从当前  $pi$  起取  $m$  个汉字作为词  $wi$
  6. if  $wi$  在词典中
  7. then 在  $wi$  后添加一个切分标志, 根据  $wi$  的长度修改指针  $pi$
  8. else
  9. 将  $wi$  从右端去掉一个字
  10. until  $pi$  指向字符串末端
- 输出: 添加切分标志后的字符串  $s$



```
text = "他是研究生物化学的"
Dict = ["他","是","研究","研究生","生物","物化","化学","学","的"]
def forword_Match(text, Dict):
    """前向最大匹配"""
    word_list = []
    pi = 0 #初始位置
    #找出字典中的最长的词的长度
    m = max([len(word) for word in Dict])
    while pi != len(text):
        n = len(text[pi:]) #当前指针到字符串末尾的长度
        if n < m:
            m = n
        for index in range(m,0,-1): #从当前 pi 起取 m 个汉字作为词
            if text[pi:pi+index] in Dict:
                word_list.append(text[pi:pi+index])
                pi = pi + index #根据词的长度修改指针pi
                break
        print('/'.join(word_list))
    forword_Match(text, Dict)

## 输出： 他/是/研究生/物化/学/的
```

## 局限性



### 1. 词典依赖性强

例子：如果词典中没有“深度学习”，句子“深度学习很重要”可能被错误地分成“深度/学习/很/重要”。

### 2. 无法处理歧义

例子：句子“研究生命科学”可能被错误地分成“研究生/命/科学”，而正确的分词应为“研究/生命/科学”。

### 3. 无法识别未登录词（如新词、专有名词、网络用语等）

新词“内卷”如果未收录在词典中，可能被错误地分成“内/卷”。

### 4. 缺乏上下文理解

例子：句子“他将来北京”可能被错误地分成“他/将来/北京”，而正确的分词应为“他/将/来/北京”。

### 5. 效率问题

例子：如果词典包含数十万词条，每次匹配都需要遍历大量词条，影响分词速度。

# 基于词典的分词

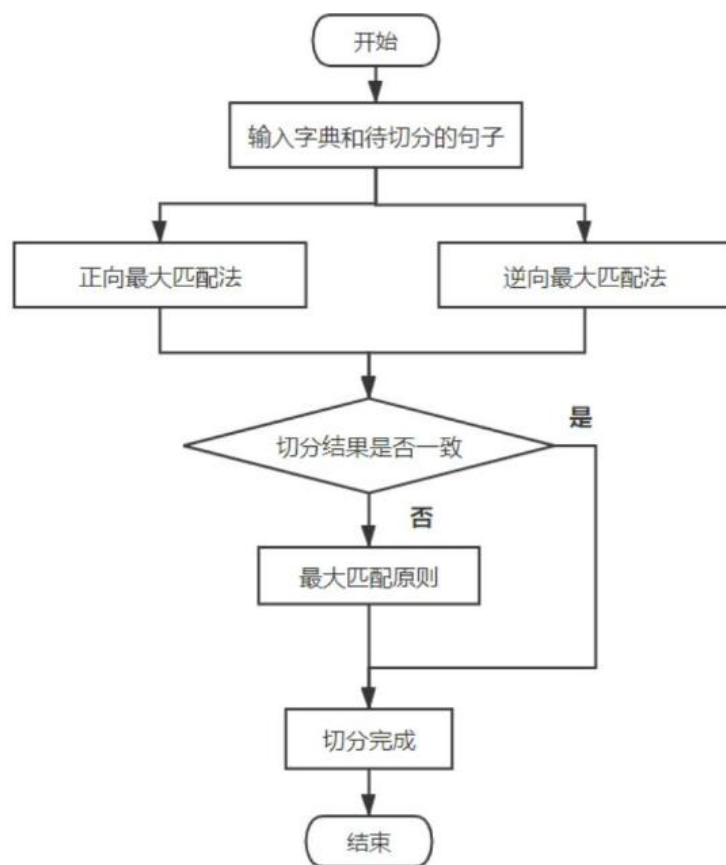
汉语中许多词的结构是“修饰语 + 中心词”，RMM比FMM更适合

## 逆向最大匹配分词(RMM)

从右到左扫描文本，尽可能匹配最长的词。

## 双向最大匹配法

动手实现下这两种算法



## 结合实例

### 知识点回顾



词输入=>语言模型=>词输出

词输入=>语言模型=>词输出

知识点回顾

## • 语言模型

衡量一句自然语言句子的概率的模型

- 单词:

$$P(\text{“谢谢”}) > P(\text{“马尔可夫”})$$

- 短语:

$$P(\text{“上课”}) > P(\text{“去课”})$$

- 句子:

$$P(\text{“今天天气真好!”}) > P(\text{“真好今天天气”})$$

# N-gram模型——N?

- 语言模型

句法、语义、语用等知识->模型参数复杂

- N-gram语言模型

N个连续的词

N=1 Unigram (Bag-of-words)

“学生”，“大学”

N=2 Bigram

“测/核酸”，“吃/零食”

N=3 Trigram

“排队/测/核酸”，“一起/吃/零食”

# Unigram语言模型

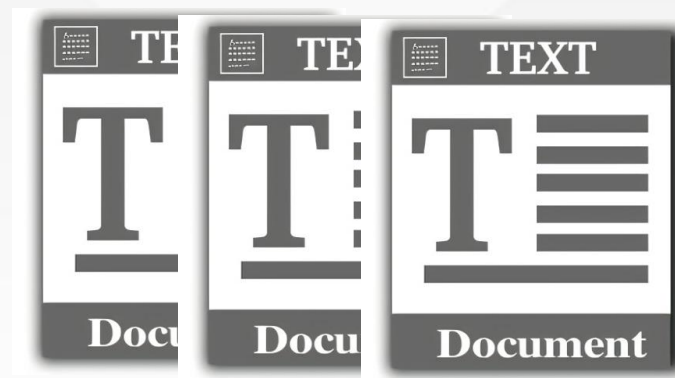
- 从掷色子到词概率模型



$$V = \{w_1, w_2, \dots, w_{|V|}\}$$

$|V|$  词典  $V$  中单词的数目

- Unigram模型
- i.i.d 词



S

创新是发展理念之首

创新 是 发展 理念 之 首

$$P(s) = P(w_1)P(w_2) \dots P(w_n)$$

$w_1$   $w_2$   $w_3$   $w_4$   $w_5$   $w_6$

# Unigram语言模型

- 缺点

未登陆词 OOV (Out-of-Vocabulary)

训练集中未出现

$$P(\text{OOV}) = 0$$

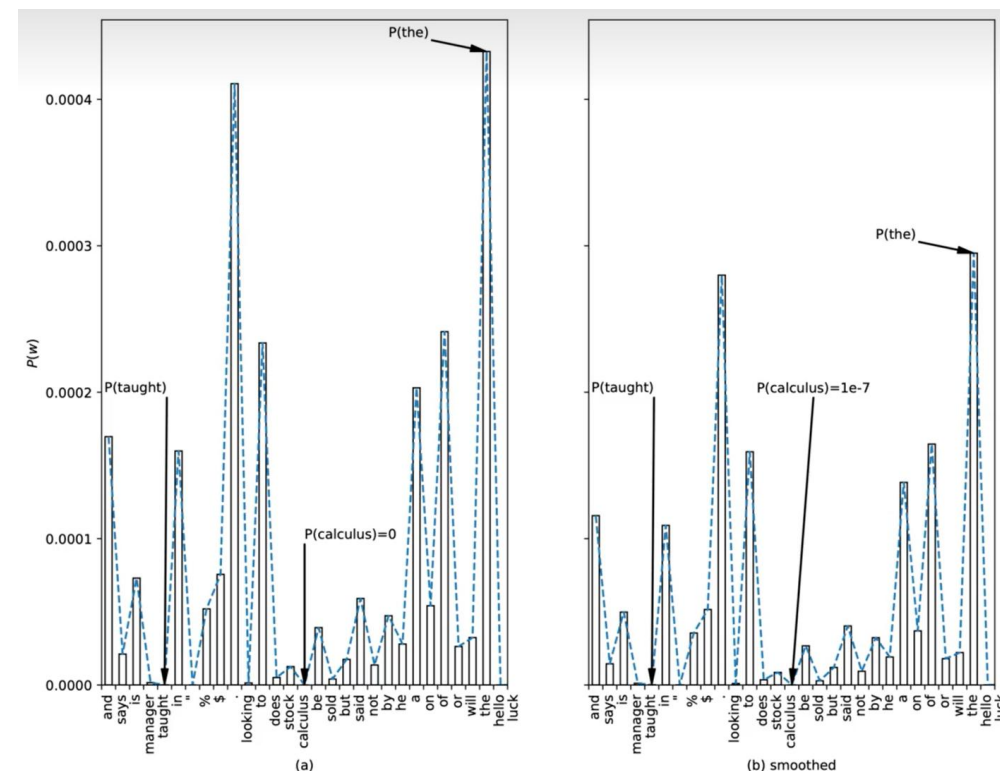
$$P(S) = 0, \text{ if OOV in } S$$

- 方法:

平滑技术

$$P(w) = \frac{(\#w \in D) + \alpha}{\sum_{w' \in V} ((\#w' \in D) + \alpha)} = \frac{(\#w \in D) + 1}{|V| + \sum_{w' \in V} (\#w' \in D)}$$

$\alpha$ 超参 add- $\alpha$  smoothing



# $n$ -gram模型概述

## $n$ -gram模型定义

定义 $n$ -gram语言模型如下：

unigram ( $n = 1$ ) :  $P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$

bigram ( $n = 2$ ) :  $P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$

trigram ( $n = 3$ ) :  $P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-2}, w_{i-1})$

更大的 $n$ ：对下一个词出现的约束信息更多，具有更大的辨别力；

更小的 $n$ ：在训练语料库中出现的次数更多，具有更可靠的统计信息，具有更高的可靠性。

理论上， $n$ 越大越好，经验上，trigram用的最多，尽管如此，原则上，能用bigram解决，绝不使用trigram。

通常，通过计算极大似然估计（Maximum Likelihood Estimate）构造语言模型，这是对训练数据的最佳估计。

公式如下：

$$p(w_{i+1} | w_{i-k}, w_i) = \frac{c(w_{i-k}, \dots, w_i)}{c(w_{i-k}, \dots, w_{i-1})}$$



jieba 是目前使用最多的Python中文分词组件，它主要有以下 3 种特性：

- 支持 3 种分词模式：
  - ✓ **精确模式**：试图将句子最精确地切开，适合文本分析；
  - ✓ **全模式**：把句子中所有的可以成词的词语都扫描出来，速度非常快，但是不能解决歧义；
  - ✓ **搜索引擎模式**：在精确模式的基础上，对长词再次切分，提高召回率，适合用于搜索引擎分词；
- 支持繁体分词；
- 支持自定义词典。

# 分词

```
1 # 全模式
2 seg_list = jieba.cut("他来到上海交通大学", cut_all=True)
3 print("【全模式】： " + "/ ".join(seg_list))
```

【全模式】： 他/ 来到/ 上海/ 上海交通大学/ 交通/ 大学

```
1 # 精确模式
2 seg_list = jieba.cut("他来到上海交通大学", cut_all=False)
3 print("【精确模式】： " + "/ ".join(seg_list))
```

【精确模式】： 他/ 来到/ 上海交通大学

```
1 # 搜索引擎模式
2 seg_list = jieba.cut_for_search("他毕业于上海交通大学机电系，后来在一机部上海电器科学研究所工作")
3 print("【搜索引擎模式】： " + "/ ".join(seg_list))
```

【搜索引擎模式】： 他/ 毕业/ 于/ 上海/ 交通/ 大学/ 上海交通大学/ 机电/ 系/ ， / 后来/ 在/ 一机部/ 上海/ 电器/ 科学/ 研究/ 研究所/ 工作

## 分词

可使用jieba.cut和jieba.cut\_for\_search方法进行分词，两者所返回的结构都是一个可迭代的generator，可使用for循环来获得分词后得到的每一个词语（unicode），或者直接使用jieba.lcut以及jieba.lcut\_for\_search直接返回list。

# 课程目录

Course catalogue

1/ 什么是词法分析

2/ 分词

3/ 关键词提取

4/ 词性标注

5/ 命名实体识别

# 什么是关键词提取

## 关键词提取的定义

**关键词提取**是从文本中抽取与内容最相关的词语，其应用可以追溯到文献检索初期。当时，由于技术限制，无法支持全文搜索，关键词便成为检索论文的重要依据。如今，关键词仍然是学术论文的标配，用于快速定位和概括文章核心内容。

除了文献检索，关键词提取在文本聚类、分类、摘要等领域也发挥着重要作用。例如：

- 文本聚类：通过提取关键词，将关键词相似的文本归为一类，可以显著提高聚类算法（如K-means）的收敛速度。
- 新闻摘要：从某天的新闻中提取关键词，可以快速了解当天的主要事件。
- 话题分析：将一段时间内多个用户的微博合并为长文本并提取关键词，可以揭示他们讨论的核心话题。

此外，关键词提取还广泛应用于搜索引擎优化（SEO）、推荐系统、知识图谱构建等领域。它不仅能够提升文本处理的效率，还能帮助用户快速捕捉文本的核心信息，是自然语言处理中的重要技术之一。

# TF-IDF

TF-IDF is a measure of originality of a word by comparing the number of times a word appears in a doc with the number of docs the word appears in.

$$\text{TF-IDF} = \text{TF}(t, d) \times \text{IDF}(t)$$

Term frequency

Number of times term  $t$  appears in a doc,  $d$

Inverse document frequency

$$\log \frac{1 + \overset{\text{\# of documents}}{n}}{1 + \underset{\text{Document frequency of the term } t}{df(d, t)}}$$

# TF-IDF

## TF-IDF

**TF-IDF (Term Frequency-Inverse Document Frequency)** 一种用于衡量词语在文档中重要性的统计方法，常用于信息检索、文本挖掘和自然语言处理领域。它结合了词频 (TF) 和逆文档频率 (IDF) 两个指标，能够有效评估一个词语在文档集合中的重要性。

词频表示一个词语在单个文档中出现的频率。词频越高，说明该词语在当前文档中越重要。

$TF(t, d)$  = 词语  $t$  在文档  $d$  中出现的次数 / 文档  $d$  中的总词语数

$$TF_{\text{smooth}} = 1 + \log(TF)$$

TF平滑的作用是什么？

答：采用对数平滑的方式可以减小高频词对文档的影响

```
1. import numpy as np
2.
3.
4. if __name__ == '__main__':
5.
6.     tf = np.array([20, 5, 2])
7.     print(tf)
8.     tf = np.log(tf)
9.     print(tf)
```

输出结果：

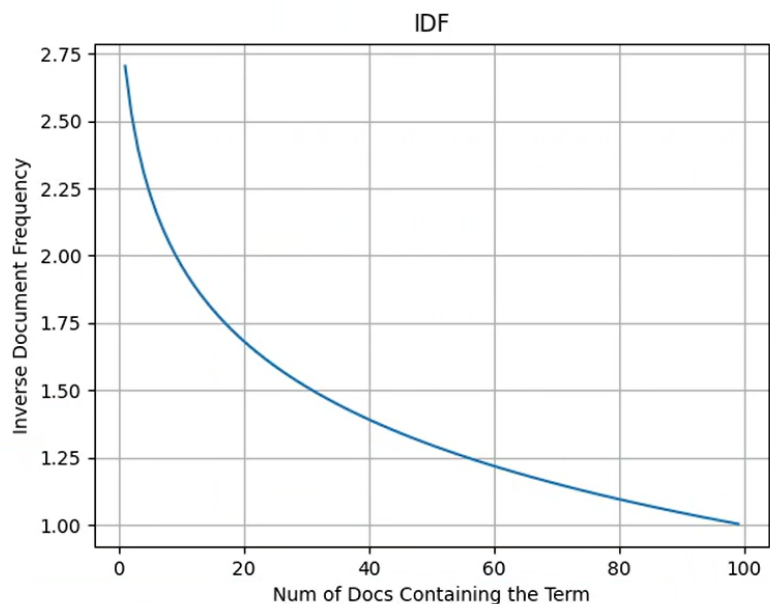
```
1. [20  5  2]
2. [2.99573227 1.60943791 0.69314718]
```

# TF-IDF

一个词的IDF是根据语料库得出的。

逆文档频率用于衡量一个词语在整个文档集中的普遍性。如果一个词语在大多数文档中都出现，则其重要性较低；反之，如果一个词语只在少数文档中出现，则其重要性较高。

$$\text{IDF}(t,D)=\log(\text{文档集合 } D \text{ 中文档总数}/\text{包含词语 } t \text{ 的文档数})$$



$$\text{IDF}(\text{逆文档词频}) = \log\left(\frac{\text{文档总数}}{\text{包含某个词的文档总数}}\right) + 1$$

$$\text{IDF}_{\text{smooth}} = \log\left(\frac{\text{文档总数} + 1}{\text{包含某个词的文档总数} + 1}\right) + 1$$

IDF平滑的作用是什么？

答：为了避免分母为0。



# TF-IDF

TF-IDF值为：TF-IDF=TF\*IDF

例如，词A和词B是某个文档中出现的词，两个词在文档中出现的次数一样多，但是

1. 词A在其他文档中大量出现
2. 词B在其他文档中很少出现

TF-IDF认为哪个词更重要？

```
1. pip install scikit-learn
```

```
1. from sklearn.feature_extraction.text import TfidfVectorizer
2.
3.
4. # 注意：文档需要先进行分词，以空格隔开
5. texts = ['我们 是 中国人 中国人', '中国人 很 很 厉害', '他们 真的 厉害 吗 ? ']
6.
7.
8. def test():
9.     tfidf = TfidfVectorizer()
10.    result = tfidf.fit_transform(texts)
11.    print(tfidf.get_feature_names_out())
12.    print(result.toarray())
13.
14.
15. if __name__ == '__main__':
16.     test()
```

TfidfVectorizer 的计算过程如下：

1. 首先，根据输入的多个文档、预料构建词表
2. 然后，统计每篇文档中，词表中的词出现的次数
3. 接着，计算每个词的平滑之后的 TF-IDF 值
4. 最后，对每个文档 TF-IDF 向量进行 L2 标准化

1. [‘中国人’，‘他们’，‘厉害’，‘我们’，‘真的’]
2. [[2 0 0 1 0], [1 0 1 0 0], [0 1 1 0 1]]
3. 以(1, 1)为例， $2 * (\log(4/3) + 1) = 2.57536414$
4. 以(1, 1)为例， $2.57 / ||x||_2$

$$||x||_2 = \sqrt{\sum_{i=1}^n x_i^2}$$



## TF-IDF

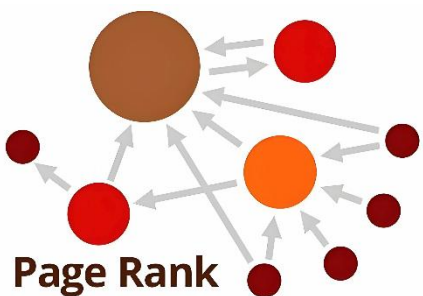
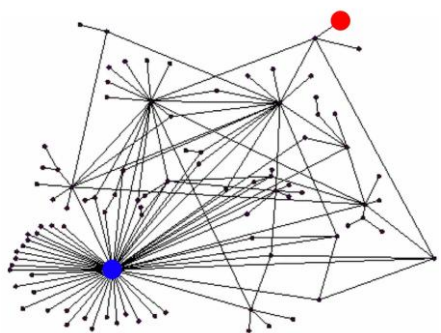
TF-IDF 是一种简单易懂的关键词提取方法，但其缺点较为明显：

- 依赖语料库：TF-IDF 的效果严重依赖语料库的质量和相关性，需要高质量且与目标文本匹配的语料库进行训练。
- 精度有限：IDF 倾向于赋予低频词较高权重，可能导致噪声词被误判为关键词，影响精度。
- 忽略位置信息：TF-IDF 无法捕捉词的位置信息（如标题、首句、尾句等），而这些位置通常包含重要信息，应赋予更高权重。

基于统计特征的关键词提取算法通过量化指标对关键词进行排序，选取 Top K 个词作为关键词。其核心在于特征量化指标的设计，不同指标各有优缺点。实际应用中，通常结合多种量化指标，以提升关键词提取的准确性和鲁棒性。

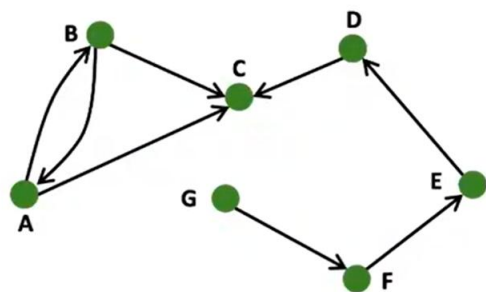
# TextRank

## TextRank



### Directed

- **Links:** directed (arcs)



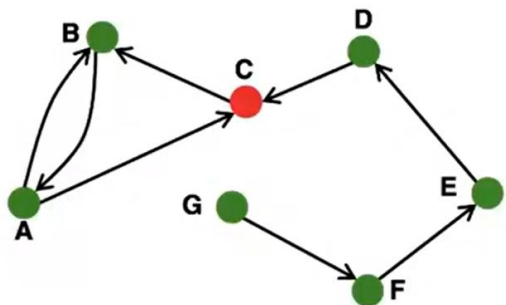
### ■ Examples:

- Phone calls
- Following on Twitter

**TextRank**算法是一种文本排序算法，由谷歌的网页重要性排序算法PageRank算法改进而来。PageRank算法将整个互联网看作一张有向图，网页是图中的节点，而网页之间的链接就是途中的边。根据重要性传递的思想，如果一个网页A含有一个指向网页B的链接，那么网页B的重要性排名会根据A的重要行来提升。

在PageRank算法中，对于网页初始重要值（PR值）的计算非常关键，但是这个值无法预知，于是PageRank论文中给出了一种迭代算法求出这个PR值：为每个网页随机给一个初始值，然后迭代得到收敛值，作为网页重要性的度量。

## Directed



**Source:** Node with  $k^{in} = 0$

**Sink:** Node with  $k^{out} = 0$

In directed networks we define an **in-degree** and **out-degree**.  
The (total) degree of a node is the sum of in- and out-degrees.

$$k_C^{in} = 2 \quad k_C^{out} = 1 \quad k_C = 3$$

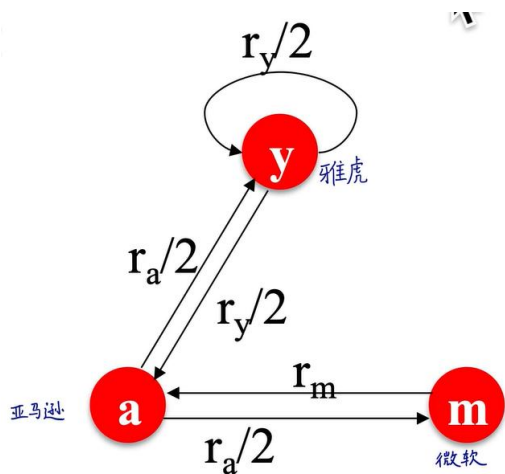
$$\bar{k} = \frac{E}{N}$$

$$\bar{k}^{in} = \bar{k}^{out}$$

## Define "rank" $r_j$ for node $j$

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

$d_i$  ... out-degree of node  $i$



$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

$$r_a + r_y + r_m = 1$$

高斯消元法

$$r_a = \frac{2}{5}$$

$$r_y = \frac{2}{5}$$

$$r_m = \frac{1}{5}$$

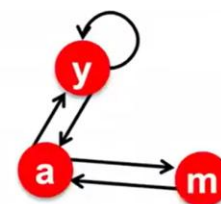
## Power Iteration:

- Set  $r_j \leftarrow 1/N$
- 1:  $r'_j \leftarrow \sum_{i \rightarrow j} \frac{r_i}{d_i}$
- 2: If  $|r - r'| > \epsilon$ :  
  - $r \leftarrow r'$
- 3: go to 1

## Example:

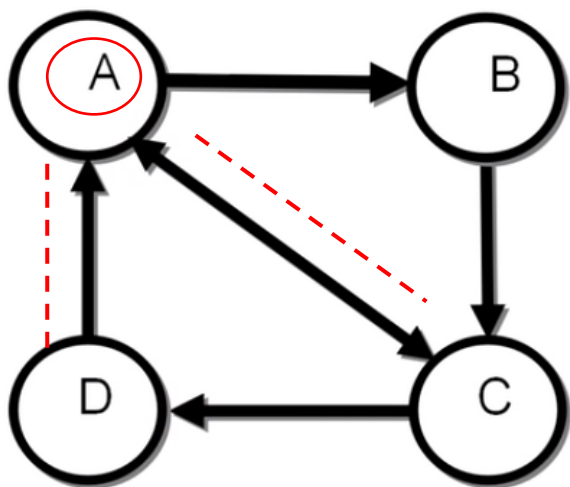
$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \quad \begin{bmatrix} 1/3 \\ 3/6 \\ 1/6 \end{bmatrix} \quad \begin{bmatrix} 5/12 \\ 1/3 \\ 3/12 \end{bmatrix} \quad \begin{bmatrix} 9/24 \\ 11/24 \\ 1/6 \end{bmatrix} \quad \dots \quad \begin{bmatrix} 6/15 \\ 6/15 \\ 3/15 \end{bmatrix}$$

Iteration 0, 1, 2, ...



|   | y   | a   | m |
|---|-----|-----|---|
| y | 1/2 | 1/2 | 0 |
| a | 1/2 | 0   | 1 |
| m | 0   | 1/2 | 0 |

$$\begin{aligned} r_y &= r_y/2 + r_a/2 \\ r_a &= r_y/2 + r_m \\ r_m &= r_a/2 \end{aligned}$$



| PR 值<br>循环次数 i           | PR (A)        | PR (B)        | PR (C)        | PR (D)        |
|--------------------------|---------------|---------------|---------------|---------------|
| i = 0<br>PR值初始化<br>= 1/N | $\frac{1}{4}$ | $\frac{1}{4}$ | $\frac{1}{4}$ | $\frac{1}{4}$ |
| i=1                      | $\frac{3}{8}$ | $\frac{1}{8}$ | $\frac{3}{8}$ | $\frac{1}{8}$ |
| 排名                       | 1             | 2             | 1             | 2             |

$$PR(A) = PR(D)/1 + PR(C)/2$$

| PR 值<br>循环<br>次数 i       | PR (A)        | PR (B)        | PR (C)        | PR (D)        |
|--------------------------|---------------|---------------|---------------|---------------|
| i = 0<br>PR值初始化<br>= 1/N | $\frac{1}{4}$ | $\frac{1}{4}$ | $\frac{1}{4}$ | $\frac{1}{4}$ |
| i=1                      | $\frac{3}{8}$ | $\frac{1}{8}$ | $\frac{3}{8}$ | $\frac{1}{8}$ |
| 排名                       | 1             | 2             | 1             | 2             |



矩阵化表达  
转移概率矩阵（也称为马尔科夫矩阵）  
是描述马尔科夫链中状态转移概率的矩阵。

定义

- **状态空间**: 马尔科夫链有有限个状态, 记为  $S = \{s_1, s_2, \dots, s_n\}$ 。
- **转移概率**: 从状态  $s_i$  转移到状态  $s_j$  的概率为  $P_{ij}$ , 满足:

$$P_{ij} \geq 0 \quad \text{且} \quad \sum_{j=1}^n P_{ij} = 1 \quad \text{对所有 } i$$

- **转移概率矩阵**: 将  $P_{ij}$  排列成矩阵  $P$ :

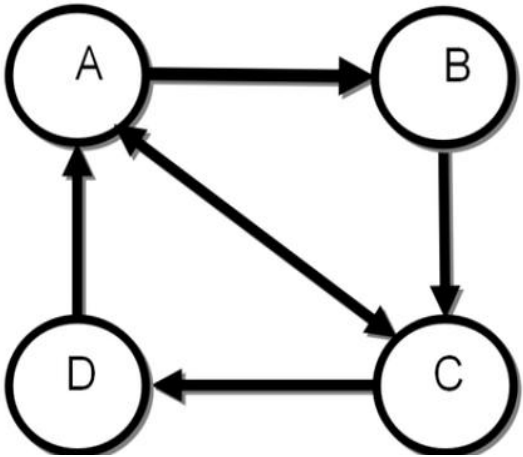
$$P = \begin{pmatrix} P_{11} & P_{12} & \dots & P_{1n} \\ P_{21} & P_{22} & \dots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \dots & P_{nn} \end{pmatrix}$$



(矩阵的列表示出链)  
从A将跳转到 B 或 C 的概率为 1/2。  
从D将跳转到 A 的概率为 1。



$$\begin{matrix} & A & B & C & D \\ \begin{matrix} A \\ B \\ C \\ D \end{matrix} & \left[ \begin{array}{cccc} 0 & 0 & 1/2 & 1 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 0 \\ 0 & 0 & 1/2 & 0 \end{array} \right] \end{matrix}$$



通过矩阵化表达，快速计算 PR 值

$$PR(a) = M * V$$

$$\begin{bmatrix} 0 & 0 & 1/2 & 1 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 0 \\ 0 & 0 & 1/2 & 0 \end{bmatrix} \times \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 3/8 \\ 1/8 \\ 3/8 \\ 1/8 \end{bmatrix} \quad (\text{第一次迭代得到的 PR 值})$$

初始化的Pr值  $1/N = 1/4$

$$\begin{array}{c} A \\ B \\ C \\ D \end{array} \begin{bmatrix} 0 & 0 & 1/2 & 1 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 0 \\ 0 & 0 & 1/2 & 0 \end{bmatrix} \times \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 3/8 \\ 1/8 \\ 3/8 \\ 1/8 \end{bmatrix} \quad \text{第一次迭代得到的PR值}$$

第二次迭代

$$\begin{array}{c} A \\ B \\ C \\ D \end{array} \begin{bmatrix} 0 & 0 & 1/2 & 1 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 0 \\ 0 & 0 & 1/2 & 0 \end{bmatrix} \times \begin{bmatrix} 3/8 \\ 1/8 \\ 3/8 \\ 1/8 \end{bmatrix} = \begin{bmatrix} 5/16 \\ 3/16 \\ 5/16 \\ 3/16 \end{bmatrix} \quad \text{第二次迭代得到的PR值}$$

# TextRank

## TextRank 的核心思想

### 1.构建图：

1. 将文本单元（如单词或句子）作为图中的节点。[\(网页\)](#)
2. 根据文本单元之间的关系（如共现、语义相似性）构建边，边的权重表示关系的强度。[\(网页之间的链接\)](#)

### 2.迭代计算节点权重：

1. 使用类似 PageRank 的公式迭代计算每个节点的重要性分数。
2. 节点的重要性取决于其邻居节点的权重和边的权重。

### 3.排序与提取：

1. 根据节点的最终权重排序，选择最重要的节点作为输出（如关键词或摘要句）。

TextRank计算公式：

$$WS(V_i) = (1 - d) + d \times \sum_{j \in In(V_i)} \frac{w_{ji}}{\sum_{V_k \in Out(V_j)} w_{jk}} WS(V_j)$$

- $WS(V_i)$ : 节点  $V_i$  的权重。
- $d$ : 阻尼系数（通常取 0.85），用于控制随机跳转的概率。
- $In(V_i)$ : 指向节点  $V_i$  的节点集合。
- $Out(V_j)$ : 从节点  $V_j$  指向的节点集合。
- $w_{ji}$ : 节点  $V_j$  到节点  $V_i$  的边权重。

TextRank



# 课后动手练习

关键词提取

```
import jieba.analyse

text = "人工智能是研究、开发用于模拟、延伸和扩展人的智能的理论、方法、技术及应用系统的一门新的技术科学"

# 基于TF-IDF 算法的关键词提取
keywords = jieba.analyse.extract_tags(text, topK=5, withWeight=True)
print("关键词提取:")
for keyword, weight in keywords:
    print(f"{keyword}: {weight:.2f}")

# 基于TextRank 算法的关键词提取
keywords = jieba.analyse.textrank(text, topK=5, withWeight=True)
print("\nTextRank 关键词提取:")
for keyword, weight in keywords:
    print(f"{keyword}: {weight:.2f}")
```

运行结果:

```
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\Administrator\AppData\Local\Temp\jieba.cache
关键词提取:
人工智能: 0.59
技术: 0.59
模拟: 0.48
一门: 0.46
智能: 0.45

TextRank 关键词提取:
技术: 1.00
开发: 0.78
模拟: 0.78
方法: 0.76
研究: 0.64
Loading model cost 0.831 seconds.
Prefix dict has been built successfully.
```

进程已结束, 退出代码为 0



# 课程目录

Course catalogue

1/ 什么是词法分析

2/ 分词

3/ 关键词提取

4/ 词性标注

5/ 命名实体识别

# 什么是词性标注

## 什么是词性标注

词性标注在自然语言处理中也属于基础性的模块，为句法分析、信息抽取等工作打下基础。和分词一样，中文词性标注也存在着很多难点，比如一词多词性，未登录词处理等诸多问题。通过基于字符串匹配的字典查询算法和基于统计的词性标注算法，可以很好的解决这些问题。一般需要先将语句进行分词，然后再进行词性标注。



# 词性标注的难点

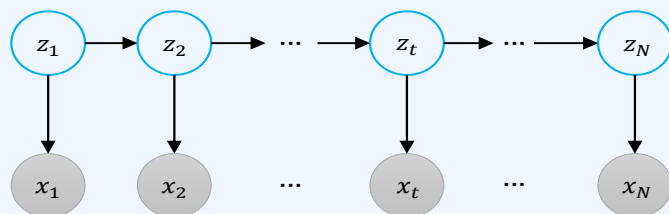
## 词性标注的难点

词性作为词语基本的语法属性，是词语和语句的关键性特征。词性种类也很多。词性标注中的难点主要有

- 相对于英文，**中文缺少词形态变化**，不能从词的形态来识别词性；
- 一词多词性很常见。统计发现，一词多词性的概率高达22.5%。而且越常用的词，**多词性现象**越严重。比如“研究”既可以是名词（“基础性研究”），也可以是动词（“研究计算机科学”）；
- **词性划分标准不统一**。词类划分粒度和标记符号等，目前还没有一个广泛认可的统一的标准。比如LDC标注语料中，将汉语一级词性划分为33类，而北京大学语料库则将其划分为26类。词类划分标准和标记符号的不统一，以及分词规范的含糊，都给词性标注带来了很大的困难。jieba分词采用了使用较为广泛的ICTCLAS 汉语词性标注集规范。
- **未登录词问题**。和分词一样，未登录词的词性也是一个比较大的课题。未登录词不能通过查找字典的方式获取词性，可以采用HMM隐马尔科夫模型等基于统计的算法。

# 隐马尔可夫模型

## Hidden Markov Model (HMM)



在任意时刻 $t$ ，给定 $z_t$ ，观测变量 $x_t$ 的取值与其他隐变量无关。同时，对于任意时刻的隐变量 $z_t$ ，如果给定其前一时刻的隐变量 $z_{t-1}$ ，则与更早时刻的隐变量没有关系。图中对应的联合概率分布为：

$$p(x_1, z_1, x_2, z_2, \dots, x_N, z_N) = p(z_1) p(x_1 | z_1) \prod_{t=2}^N p(z_t | z_{t-1}) p(x_t | z_t).$$

概率模型基础

两大假设 & 三参数

Viterbi 算法

NLP 实例 & 练习

# 背景：生成模型 vs 判别模型

HMM 是生成式概率模型——先学联合分布，再做推断

## 生成模型 Generative

建模目标：学习联合分布  $P(O, Y)$

推断方式：用贝叶斯法则转化为  $P(Y|O)$

典型代表：朴素贝叶斯、HMM

特点：可以生成新样本；对分布建模更完整

## 判别模型 Discriminative

建模目标：直接学习条件分布  $P(Y|O)$

推断方式：直接计算，无需贝叶斯转化

典型代表：逻辑回归、CRF

特点：分类任务更直接；特征工程更灵活

★ HMM 是生成模型，本节重点讲解

# HMM 定义与图结构

两组变量 + 马尔可夫链 + 观测独立

## 核心定义

### 状态变量

$\{s_1, s_2, \dots, s_T\}$ ,  $s_t \in S$   
表示  $t$  时刻系统的隐藏状态  
(不可观测)

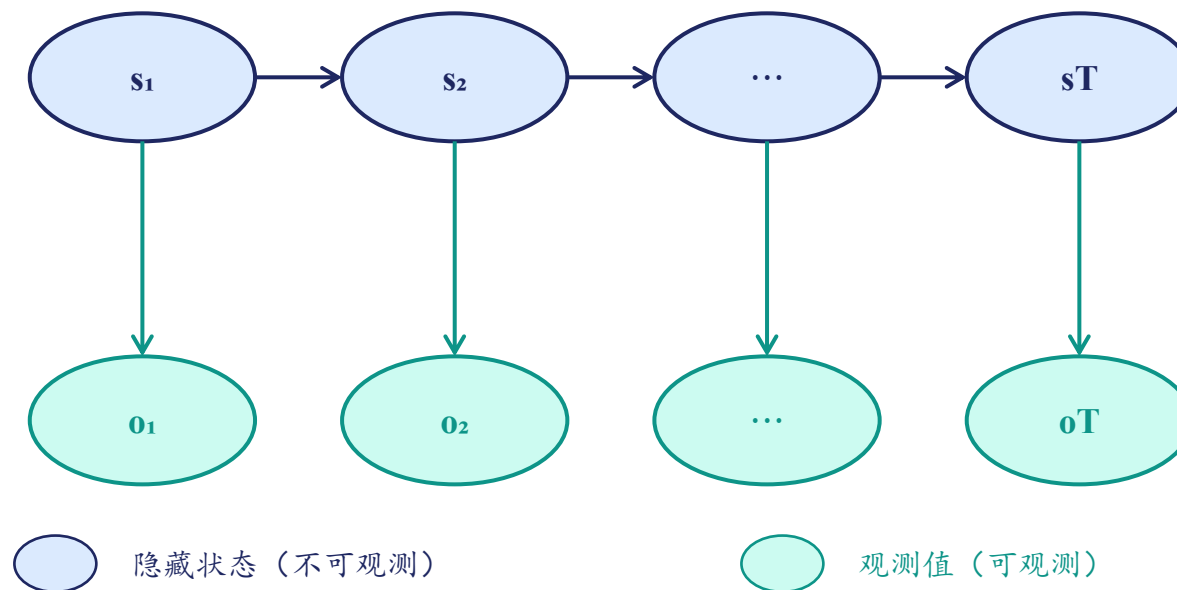
### 观测变量

$\{o_1, o_2, \dots, o_T\}$ ,  $o_t \in O$   
表示  $t$  时刻的观测值  
(可观测)

### 生成过程

隐藏状态由马尔可夫链生成；  
每个状态再随机发射出一个观测值

## 贝叶斯网络图结构



**关键：** HMM 中 H (Hidden) 因为状态序列是隐藏的，只有观测序列可见，算法目标是从观测推断隐状态。

# 两大假设 + 三组参数

$\lambda = (A, B, \pi)$  确定一个 HMM

## 两大核心假设

### 假设 1 齐次马尔可夫假设

t 时刻的状态只依赖于 t-1 时刻的状态，与其他时刻的状态及观测无关：

$$P(s_t | s_{t-1}, \dots, s_1, o_1) = P(s_t | s_{t-1})$$

### 假设 2 观测独立性假设

t 时刻的观测值只依赖于该时刻的状态，与其他观测及状态无关：

$$P(o_t | s_t, \dots, s_1, o_1, \dots) = P(o_t | s_t)$$

## 三组参数 $\lambda = (A, B, \pi)$

### A 状态转移矩阵

$$a_{ij} = P(s_{t+1}=S_j | s_t=S_i)$$

从状态  $S_i$  转移到  $S_j$  的概率

矩阵大小：N × N

### B 发射概率矩阵

$$b_{ij} = P(o_t=O_j | s_t=S_i)$$

处于状态  $S_i$  时观测到  $O_j$  的概率

矩阵大小：N × M

### $\pi$ 初始状态概率

$$\pi_i = P(s_1=S_i)$$

初始时刻处于各状态的概率

长度：N 维向量

# HMM 的三大基本问题

给定不同已知信息，分别要解决什么？

01

## 概率计算问题（Evaluation）

Forward/Backward 算法

已知：模型  $\lambda = [A, B, \pi]$  + 观测序列  $O$

目标：计算观测序列  $O$  在模型  $\lambda$  下出现的概率  $P(O|\lambda)$

*NLP例：*「我爱 NLP」这句话在模型下出现的概率是多少？

02

## 预测问题——解码（Decoding）

Viterbi 算法 ★

已知：模型  $\lambda = [A, B, \pi]$  + 观测序列  $O$

目标：找使  $P(O|S)$  最大的状态序列  $S^*$ （最可能的隐藏路径）

*NLP例：*「我爱 NLP」每个词最可能的词性是什么？

03

## 学习问题（Learning/Training）

Baum-Welch / EM

已知：观测序列  $O$ （或加部分标注  $S$ ）

目标：估计最优参数  $\lambda^* = \operatorname{argmax} P(O|\lambda)$

*NLP例：*从大量已标注语料中统计参数  $A$ 、 $B$ 、 $\pi$



# Viterbi 算法——解码问题的解法

动态规划找最优路径，核心公式： $\delta_t(j) = \max[\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \times a_{ij}] \times b_j(o_t)$$

$\delta_t(j)$ : t时刻到达状态j的最大路径概率     $a_{ij}$ : 转移概率     $b_j(o_t)$ : 发射概率

## Step 1 初始化

t=1 时刻:  
 $\delta_1(i) = \pi_i \cdot b_i(o_1)$   
初始得分 = 初始概率 ×  
发射概率

## Step 2 递推

t=2,...,T:  
 $\delta_t(j) =$   
 $\max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$   
同时记录最优前驱  $\psi_t(j)$

## Step 3 终止

找终止时刻最高分状态:  
 $P^* = \max_i \delta_T(i)$   
 $i^* = \operatorname{argmax} \delta_T(i)$

## Step 4 回溯

从  $i_T^*$  反向追踪:  
 $s_t^* = \psi_{t+1}(s_{t+1}^*)$   
输出最优状态序列  $S^*$

时间复杂度:  $O(T \cdot N^2)$ , 远优于暴力枚举  $O(N^T T)$

# 生活实例：天气预测

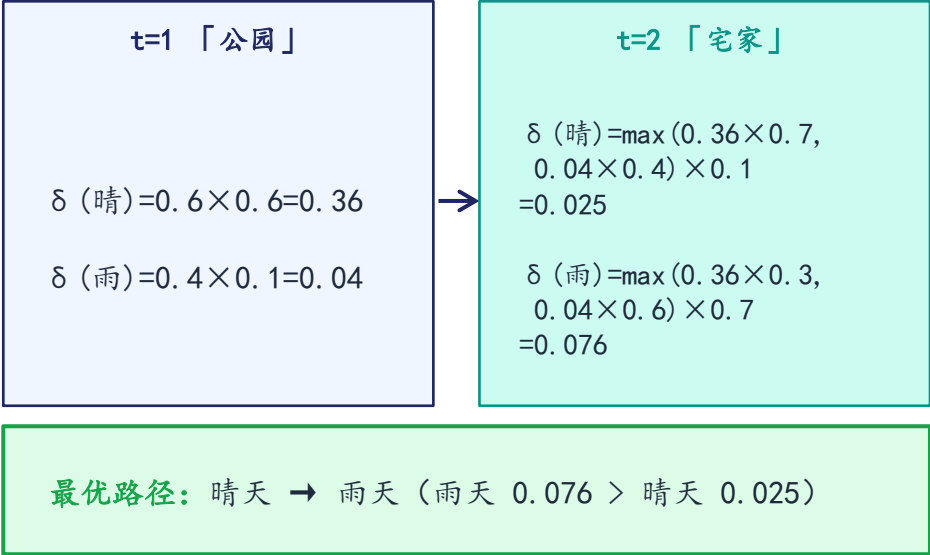
你在珀斯，看小明的朋友圈行为，反推北京的天气

小明每天发朋友圈：公园 / 宅家 / 购物。你在珀斯看不到北京的天气，但想从他的行为推断这几天是晴还是雨。

## HMM 对应关系

| HMM 概念     | 天气实例                                            |
|------------|-------------------------------------------------|
| 隐藏状态 S     | 天气：{晴天, 雨天}                                     |
| 观测变量 O     | 行为：{公园, 宅家, 购物}                                 |
| 转移概率 A     | 晴→晴=0.7, 晴→雨=0.3; 雨→晴=0.4, 雨→雨=0.6              |
| 发射概率 B     | 晴:公园=0.6,宅家=0.1,购物=0.3 / 雨:公园=0.1,宅家=0.7,购物=0.2 |
| 初始概率 $\pi$ | $\pi(\text{晴})=0.6, \pi(\text{雨})=0.4$          |

## Viterbi 解码：观测「公园 → 宅家」



**直觉验证：**第1天去公园→晴天可能大（晴天去公园概率0.6）；第2天宅家→很可能下雨（雨天宅家0.7远大于晴天0.1）  
所以模型判断晴→雨，与我们的直觉完全一致！

# HMM 在 NLP 中的应用：词性标注

天气预测 vs 词性标注——同一个问题的两种表述

| 维度     | 天气预测             | 词性标注 (POS Tagging)          |
|--------|------------------|-----------------------------|
| 隐藏状态 S | 晴天 / 雨天          | 名词(n) / 动词(v) / 代词(r) / ... |
| 观测序列 O | 公园 / 宅家 / 购物     | 「我」 / 「爱」 / 「NLP」           |
| 转移概率 A | 晴→晴=0.7, 晴→雨=0.3 | n→v=0.5, v→n=0.6, ...       |
| 发射概率 B | 晴→公园=0.6         | n→「NLP」=0.8                 |
| 目标问题   | 看行为, 猜天气         | 看词, 猜词性                     |
| 解码算法   | Viterbi          | Viterbi (完全相同!)             |

## jieba 实现 (HMM + Viterbi 内置)

```
import jieba.posseg as pseg
text = "我爱自然语言处理技术"
for w, flag in pseg.cut(text):
    print(f'{w}/{flag}')
```

## 输出结果

```
# 我/r 爱/v 自然语言/l 处理/vn 技术/n
# jieba 内部使用 HMM + Viterbi 解码
# 每个词的词性 = 最优隐状态序列
```

# 课堂练习题 — Viterbi 词性标注计算

给定参数，求「我爱 NLP」的最优词性标注路径

## 已知 HMM 参数

状态集  $S = \{ n(\text{名词}), v(\text{动词}), r(\text{代词}) \}$

初始概率  $\pi$ :

$r=0.6 \quad n=0.3 \quad v=0.1$

转移概率  $A$  (行=前一状态, 列=当前):

|   | n               | v | r |
|---|-----------------|---|---|
| n | [0.2, 0.5, 0.3] |   |   |
| v | [0.6, 0.1, 0.3] |   |   |
| r | [0.1, 0.7, 0.2] |   |   |

发射概率  $B$  (行=状态, 列=词):

|   | 我               | 爱 | NLP |
|---|-----------------|---|-----|
| n | [0.1, 0.1, 0.8] |   |     |
| v | [0.1, 0.8, 0.1] |   |     |
| r | [0.8, 0.1, 0.1] |   |     |

## 请填写 Viterbi 计算表格

| 状态 | t=1 「我」                 | t=2 「爱」 | t=3 「NLP」 |
|----|-------------------------|---------|-----------|
| n  | $0.3 \times 0.1 = 0.03$ | —       | —         |
| v  | —                       | —       | —         |
| r  | —                       | —       | —         |

最优路径  $S^* = \_\_\_ \rightarrow \_\_\_ \rightarrow \_\_\_$

## 思考题

- Q1:**  $r \rightarrow v \rightarrow n$  符合你的语言直觉吗? 为什么?
- Q2:** 若  $\pi(r)$  变为 0.1, 最优路径会改变吗?
- Q3:** Viterbi 算法在本题中的时间复杂度是多少?

提示:  $t=1$  得分  $= \pi_i \times b_i(o_1)$ ;  $t>1$  得分  $\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$

# 参考答案

Viterbi 完整计算过程

参考答案

| 状态 | t=1 「我」                    | t=2 「爱」                                                                                                     | t=3 「NLP」 ★                                                                                                           |
|----|----------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| n  | $0.3 \times 0.1 = 0.030$   | $\max(0.030 \times 0.2, 0.010 \times 0.6, 0.480 \times 0.1) \times 0.1$<br>$= 0.0288 \times 0.1 = 0.00288$  | $\max(0.00288 \times 0.2, 0.03456 \times 0.6, 0.000864 \times 0.1) \times 0.8$<br>$= 0.020736 \times 0.8 = 0.01659$ ★ |
| v  | $0.1 \times 0.1 = 0.010$   | $\max(0.030 \times 0.5, 0.010 \times 0.1, 0.480 \times 0.7) \times 0.8$<br>$= 0.336 \times 0.8 = 0.03456$ ★ | $\max(0.00288 \times 0.5, 0.03456 \times 0.1, 0.000864 \times 0.3) \times 0.1$<br>$= 0.003456 \times 0.1 = 0.000346$  |
| r  | $0.6 \times 0.8 = 0.480$ ★ | $\max(0.030 \times 0.3, 0.010 \times 0.3, 0.480 \times 0.2) \times 0.1$<br>$= 0.096 \times 0.1 = 0.00864$   | $\max(0.00288 \times 0.3, 0.03456 \times 0.3, 0.000864 \times 0.2) \times 0.1$<br>$= 0.010368 \times 0.1 = 0.001037$  |

最优路径  $S^* = r$  (代词)  $\rightarrow v$  (动词)  $\rightarrow n$  (名词) 即: 「我/r 爱/v NLP/n」

- Q1:** 完全符合直觉! 「我」是代词, 「爱」是动词, 「NLP」是名词, 这是标准汉语语法结构。
- Q2:** 会改变。 $\pi(r)$ 降低后 t=1 时 r 得分下降, 路径可能变为  $n \rightarrow v \rightarrow n$ 。 **Q3:**  $O(T \cdot N^2) = O(3 \times 9) = O(27)$ , 多项式时间, 暴力枚举需要  $O(N^T) = O(27)$ , 规模小时相同, 一般情况差距极大。

eg

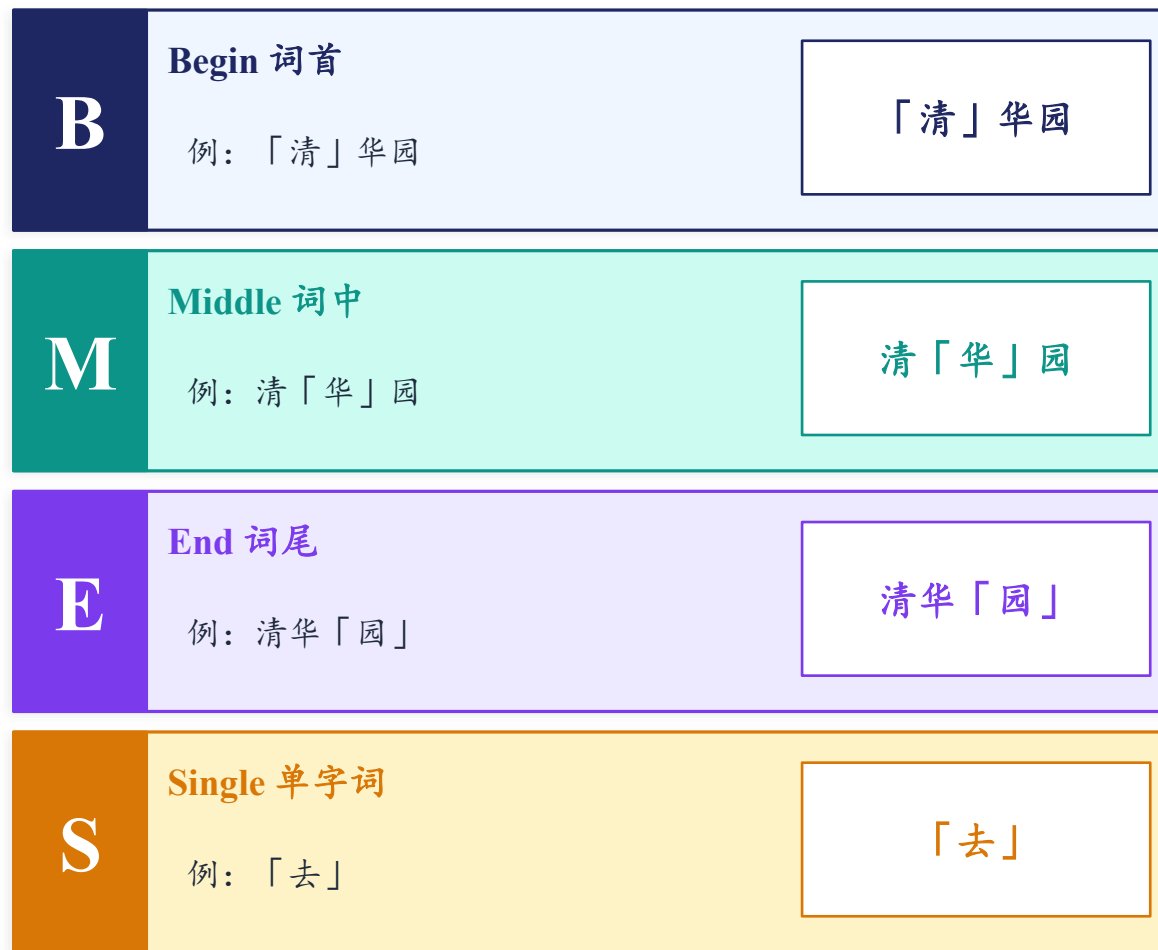
# jieba HMM: 处理 OOV 与 BEMS 状态标注

词典匹配失败时，HMM 接管——把分词变成序列标注问题

## ① HMM 的核心作用：处理 OOV



## ② BEMS 四种位置状态



合法转移：B→M、B→E、M→M、M→E、E→B、E→S、S→B、S→S | 不合法：O→I、B→B 等

# 完整实例演示

「小明去了清华园」，假设「小明」和「清华园」都不在词典里

**目标：**「小明去了清华园」中，「小明」和「清华园」均为 OOV，HMM 用 BEMS 序列逐字推断词语边界

Step 1 逐字展开

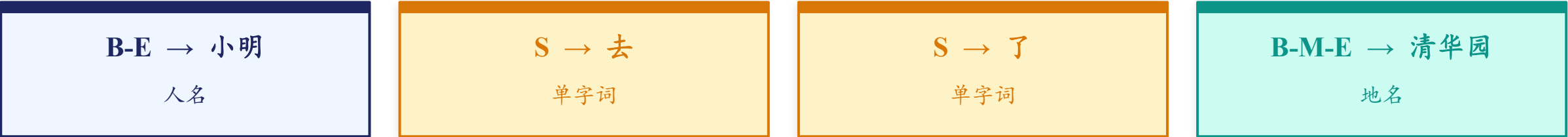


Step 2 HMM + Viterbi 分配最优 BEMS 标签



状态序列：            B   -   E   -   S   -   S   -   B   -   M   -   E

Step 3 根据 BEMS 还原词语



# HMM 分词的价值与局限

在 jieba 中仍是 OOV 处理的核心，但深度学习已全面超越

## 价值



无监督新词发现：无需人工标注，从语料自动学习边界规律



OOV 鲁棒性：人名、地名、专业术语均可处理



轻量高效：BEMS 仅 4 个状态，Viterbi 速度快



可解释性强：每个决策都有概率依据，易于调试

## 局限性



仍依赖训练语料：训练集外的新词发射概率趋近 0



上下文窗口短：马尔可夫假设只看前一状态



歧义消解弱：同一字串不同语义难以区分



已被深度学习超越：BERT-CRF 准确率全面领先

演进路线：词典规则 → HMM (jieba) → CRF → BiLSTM-CRF → BERT-CRF (当前最优)



# 课程目录

Course catalogue

1/ 什么是词法分析

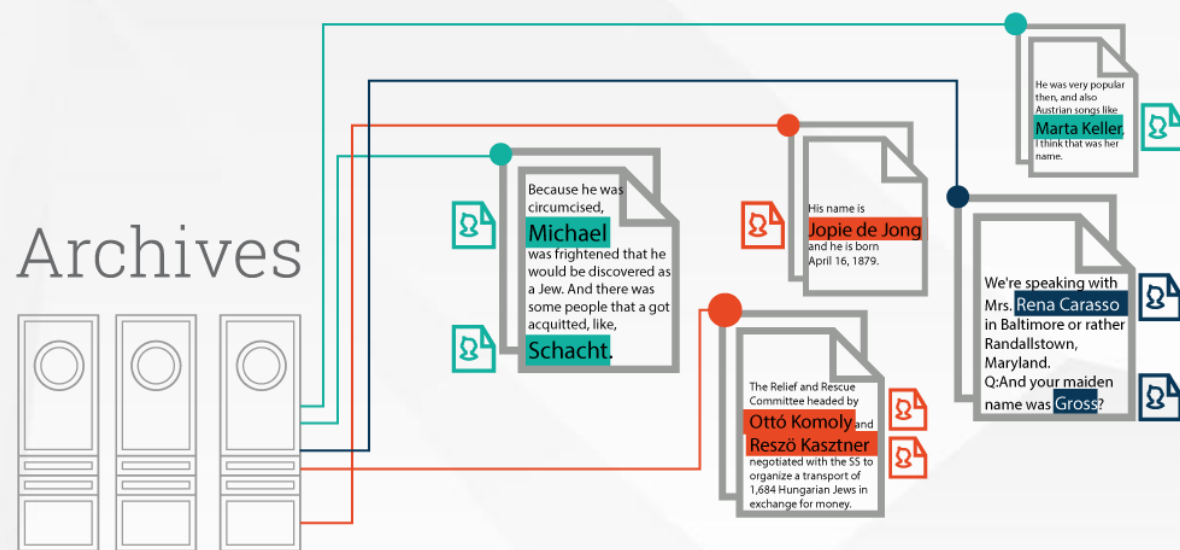
2/ 分词

3/ 关键词提取

4/ 词性标注

5/ 命名实体识别

# 什么是命名实体识别



## 什么是命名实体识别

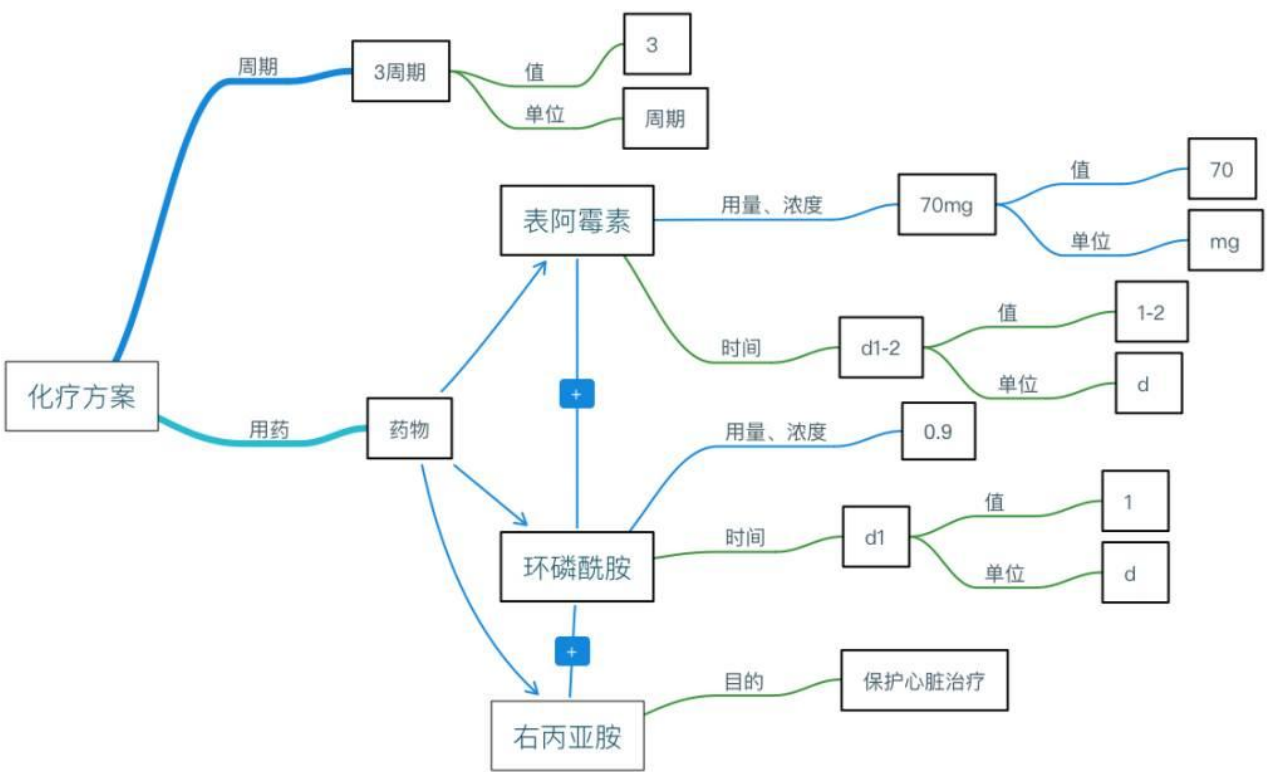
命名实体识别（Named Entity Recognition, NER）是NLP里的一项很基础的任务，就是指从文本中识别出命名性指称项，为关系抽取等任务做铺垫。狭义上，是识别出人名、地名和组织机构名这三类命名实体（时间、货币名称等构成规律明显的实体类型可以用正则表达式等方式识别）。当然，在特定的领域中，会相应地定义领域内的各种实体类型。

# 中文命名实体识别的难点

## 中文命名实体识别的难点

汉语作为象形文字，相比于英文等拼音文字来说，针对中文的NER任务来说往往要更有挑战性，下面列举几点

- ① 中文文本里不像英文那样有空格作为词语的界限标志，而且“词”在中文里本来就是一个很模糊的概念，中文也不具备英文中的字母大小写等形态指示
- ② 中文的用字灵活多变，有些词在脱离上下文语境的情况下无法判断是否是命名实体，而且就算是命名实体，当其处在不同的上下文语境下也可能是不同的实体类型
- ③ 命名实体存在嵌套现象，而且这种现象在组织机构名中尤其严重
- ④ 中文里广泛存在简化表达现象，乃至简化表达构成的命名实体。



# 条件随机场

Conditional Random Field (CRF)

算法原理 • 特征函数 • NLP 实例 • 课堂练习

HMM vs CRF

特征函数设计

条件概率公式

实例 & 练习

# 从 HMM 到 CRF——为什么需要判别模型？

HMM 生成联合分布；CRF 直接建模条件分布  $P(Y|X)$

| 维度     | HMM             | CRF          |
|--------|-----------------|--------------|
| 模型类型   | 生成式 $P(X, Y)$   | 判别式 $P(Y X)$ |
| 观测独立假设 | 需要（每个观测只依赖当前状态） | 不需要，可利用任意上下文 |
| 特征能力   | 只能用「状态→观测」一种特征  | 可定义任意特征函数    |
| 典型应用   | 分词、词性标注（基础）     | NER、词性标注（主流） |

## CRF 核心思想

### ① 直接建模 $P(Y|X)$

不关心观测序列  $X$  怎么生成，只关心给定  $X$  时标签序列  $Y$  的概率——这让特征设计更自由。

### ② 特征函数加权求和

定义一组特征函数  $f_k(y_{t-1}, y_t, X, t)$ ，每个特征赋权重  $\lambda_k$ ，所有触发特征的权重之和决定得分。

### ③ 全局归一化

对所有可能的标签序列做归一化（配分函数  $Z(X)$ ），保证概率之和为 1，同时考虑全局最优。

# CRF 核心公式与特征函数

$P(Y|X) = (1/Z(X)) \times \exp(\sum \lambda_k f_k)$ ——特征函数是 CRF 的灵魂

$$P(Y|X) = (1 / Z(X)) \times \exp(\sum_k \lambda_k \cdot f_k(y_{t-1}, y_t, X, t))$$

$Z(X)$ : 配分函数 (归一化分母)

$\lambda_k$ : 特征权重 (训练学习)

$f_k$ : 特征函数 (人工设计, 返回 0 或 1)

## 三类特征函数

### 状态特征

Emission Feature

$$f_1: \text{word}=\text{"谷歌"} \wedge \text{tag}=\text{B-ORG} \rightarrow 1$$

意图: 当前词与当前标签的匹配程度

### 转移特征

Transition Feature

$$f_2: \text{prev}=\text{B-ORG} \wedge \text{cur}=\text{I-ORG} \rightarrow 1$$

意图: 相邻标签的合法性约束

### 上下文特征

Context Feature

$$f_3: \text{prev\_word}=\text{"在"} \wedge \text{tag}=\text{B-LOC} \rightarrow 1$$

意图: HMM 做不到! 前后词对标签的影响

### 词形特征

Morphology Feature

$$f_4: \text{suffix}=\text{"大学"} \wedge \text{tag}=\text{I-ORG} \rightarrow 1$$

意图: 词的形态特征辅助判断实体类型

# NLP 实例：用 CRF 做命名实体识别

句子「李明代表清华大学在上海参加了百度举办的比赛」

**目标：**识别句中的人名（PER）、地名（LOC）、机构名（ORG），并输出 BIO 标注序列

|       |       |   |   |       |       |       |       |   |       |       |      |
|-------|-------|---|---|-------|-------|-------|-------|---|-------|-------|------|
| 李     | 明     | 代 | 表 | 清     | 华     | 大     | 学     | 在 | 上     | 海     | ...  |
| B-PER | I-PER | O | O | B-ORG | I-ORG | I-ORG | I-ORG | O | B-LOC | I-LOC | (后续) |

CRF 如何判断「清」应标 B-ORG？

$f_1$ : word=「清」  $\wedge$  tag=B-ORG

+3.0

意图：字典中「清」常做机构名词首

$f_3$ : next\_word=「华」  $\wedge$  tag=B-ORG

+1.8

意图：后接「华」时更可能是 ORG 起始

$f_2$ : prev\_tag=O  $\wedge$  tag=B-ORG

+2.0

意图：O 后接 B-ORG 是合法转移

$f_4$ : word=「清」  $\wedge$  tag=B-LOC

-1.5

意图：「清」做地名词首概率较低

总得分 (B-ORG) = 1.0 + 3.0 + 2.0 + 1.8 = 7.8 > B-LOC (低)  $\approx$  0 (低)  $\rightarrow$  「清」标注为 B-ORG  $\checkmark$

## 课堂练习题

给定特征函数与权重，完成以下三个任务

句子：「张伟在北京大学工作」 任务：CRF 命名实体识别，识别 PER / LOC / ORG

## Task A BIO 标注（每空 2 分，共 14 分）

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 张 | 伟 | 在 | 北 | 京 | 大 | 学 | 工 | 作 |
| — | — | — | — | — | — | — | — | — |

提示：识别人名、地名或机构名，其余为 0

## Task B 设计特征函数（每条 5 分，共 15 分）

请针对本句设计 3 个特征函数，  
需覆盖：状态特征、转移特征、上下文特征各 1 个。

$f_1$  (状态): \_\_\_\_\_

$f_2$  (转移): \_\_\_\_\_

$f_3$  (上下文): \_\_\_\_\_

## Task C 局部得分计算（11 分）

已知特征权重：

$f_1$ : word="北"  $\wedge$  tag=B-LOC

权重 +2.5: prev\_tag=0  $\wedge$  tag=B-ORG

权重 +1.8

$f_2$ : word="北"  $\wedge$  tag=B-ORG

权重 +1.5: next\_word="京"  $\wedge$  tag=B-LOC

权重 +2.2

$f_3$ : prev\_tag=0  $\wedge$  tag=B-LOC

权重 +2.0

问：当 prev\_tag=0, word="北", next\_word="京" 时， $\delta$  (B-LOC) 和  $\delta$  (B-ORG) 各为多少？哪个更优？



## Task A BIO 标注答案

|       |       |   |       |       |       |       |   |   |
|-------|-------|---|-------|-------|-------|-------|---|---|
| 张     | 伟     | 在 | 北     | 京     | 大     | 学     | 工 | 作 |
| B-PER | I-PER | O | B-ORG | I-ORG | I-ORG | I-ORG | O | O |

注：「北京大学」是机构名（ORG）而非地名（LOC）——“在”后接的是机构，不是单纯地点！

## Task B 特征函数设计答案

状态

$$f_1: \text{word} \in \{\text{张, 王, 李, 刘...}\} \wedge \text{tag} = \text{B-PER} \rightarrow 1$$

意图：姓氏词典特征，「张」是常见姓  
→ B-PER 强信号

转移

$$f_2: \text{prev\_tag} = \text{B-PER} \wedge \text{cur\_tag} = \text{I-PER} \rightarrow 1$$

意图：实体内部连续，B-PER 后合法接  
I-PER → 鼓励连续标注

上下文

$$f_3: \text{prev\_word} = \text{“在”} \wedge \text{tag} = \text{B-ORG} \rightarrow 1$$

意图：「在+机构名」是强触发模式，  
「在北京大学」暗示 ORG

## Task C 局部得分计算答案

$$\delta(\text{B-LOC}) = 2.5 + 2.0 + 2.2 = 6.7$$

$$f_1(+2.5) + f_3(+2.0) + f_5(+2.2)$$

$$\delta(\text{B-ORG}) = 1.5 + 1.8 = 3.3$$

$$f_2(+1.5) + f_4(+1.8) \quad (\text{无 } f_3 \text{ 加成})$$

结论：B-LOC 得分 6.7 > B-ORG 得分 3.3，模型倾向标为 B-LOC。✖ 但正确答案应为 B-ORG！说明这批特征函数设计不够完善，需补充「大学后缀→ORG」等特征。

---

谢谢